

CINECA AGENTIC PLATFORM

An Enterprise-Grade Multi-Tenant AI Orchestrator with MCP Tools and Secure Graph Querying

University:

Sapienza University of Rome

Department:

Information Engineering, Electronics and Telecommunications

Academic Year:

2025/2026

University Advisor:

Prof. Maria De Marsico

Company:

CINECA

Author:

Arman Feili

University Co-Advisors:

Prof. Marco Raoul Marini

Dr. Valerio Venanzi

Cineca Advisors:

Dr. Giuseppe Melfi

Dr. Marco Puccini



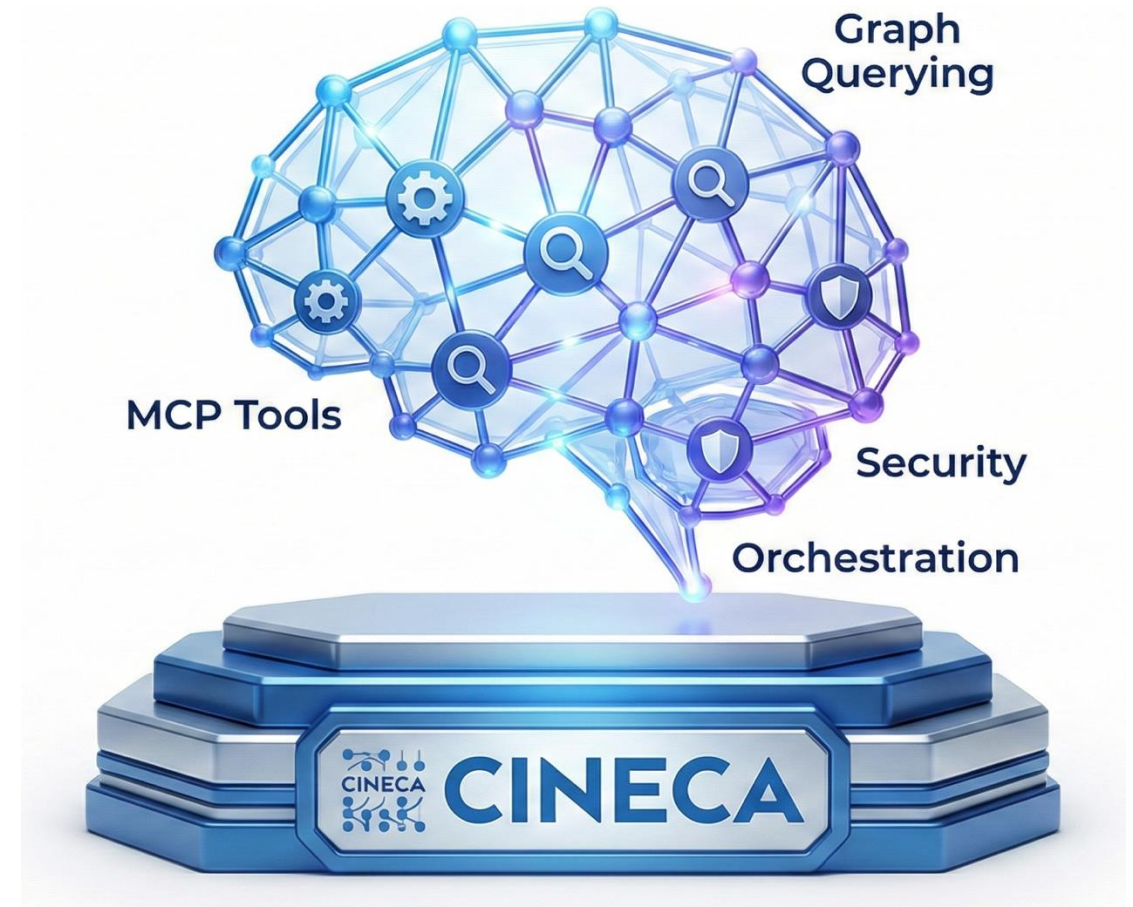
All rights relating to this teaching material and its contents are reserved by Sapienza and its authors (or teachers who produced it). Personal use of the same by the student for study purposes is permitted. Its dissemination, duplication, assignment, transmission, distribution to third parties or to the public is absolutely prohibited under penalty of the sanctions applicable by law.

CINECA

TABLE OF CONTENT

Overview of the presentation structure

1. Problem and Motivation
2. Platform Architecture
3. Core Execution Workflow
4. Security, Tools, and Data Layer
5. Validation and Comparison
6. Conclusion and Future Work



PROBLEM & MOTIVATION

Production gap

The problem

- Users need natural-language access to graph data, but current tools are too technical and not secure.
- Most users cannot write Cypher to access Graph data.
- Researchers often depend on developers to access it.
- This creates delays and keeps data locked.

Simple chatbots lack

- Multi-step execution
- Secure database and tool access
- RBAC and tenant isolation
- Audit logs and reproducible results

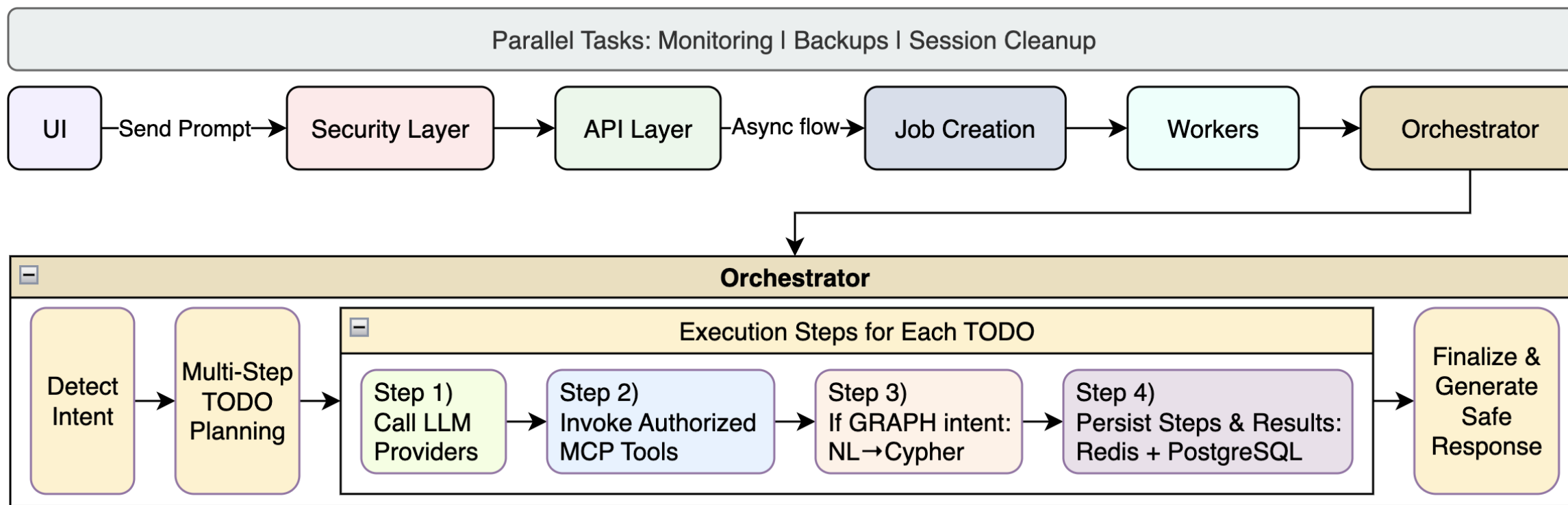


Solution

- CINECA needs a platform that turns natural-language requests into **secure, traceable, and reproducible graph actions**, while staying scalable and flexible across LLM providers.

SYSTEM WORKFLOW

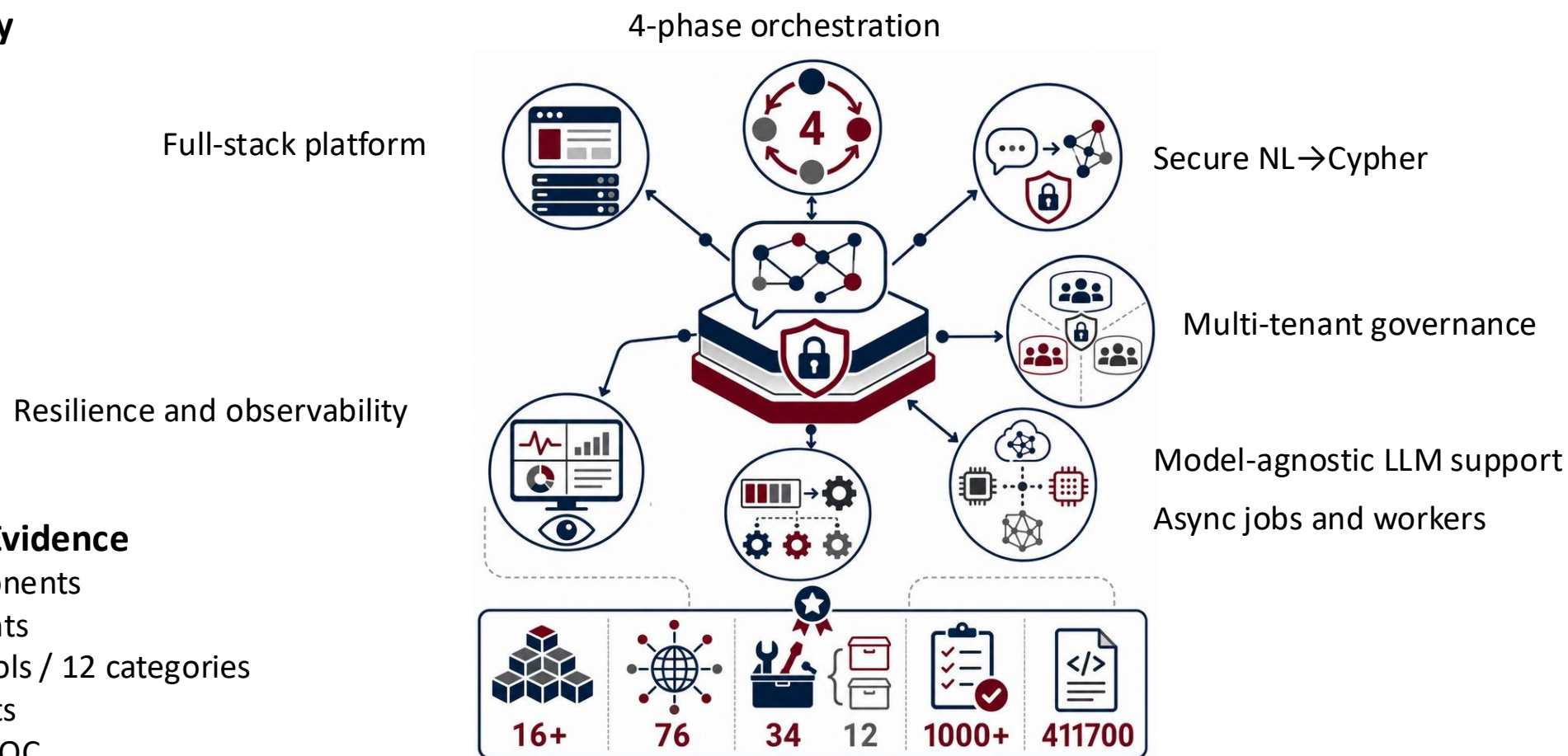
Platform Architecture



WHAT WAS DELIVERED

A production-ready agentic platform for secure natural-language access to graph data.

Core Delivery



Production Evidence

- 16+ components
- 76 endpoints
- 34 MCP tools / 12 categories
- 1,000+ tests
- ~411,700 LOC

UI

Admin & User chatbot UI

Main Flow

Users sign in through an identity provider, receive a JWT, and access the platform through a secure gateway.

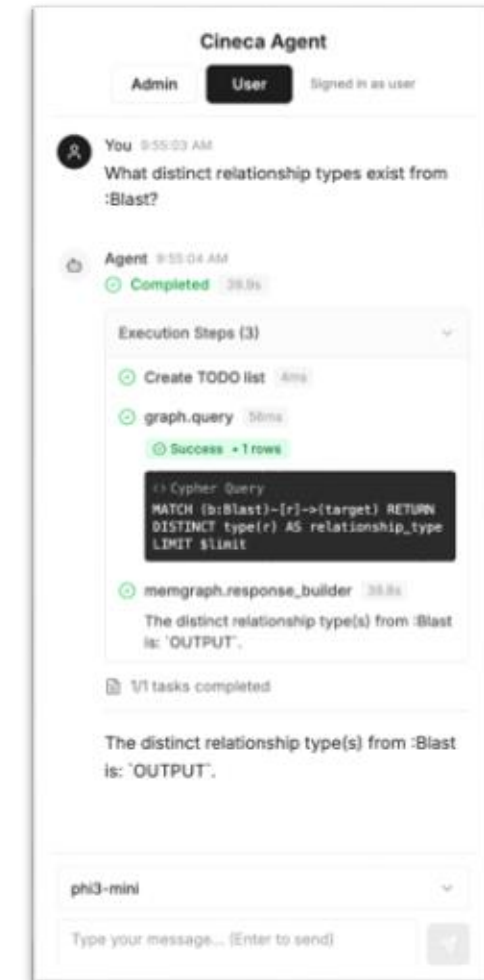
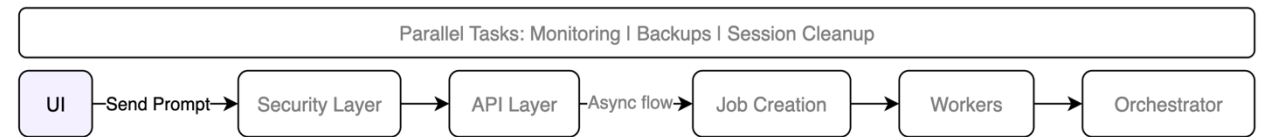
Agent Chat UI

- Built with Next.js
- Natural-language interaction
- Authenticated backend requests
- Live progress and execution steps
- Final results and selected model

Identity & Access

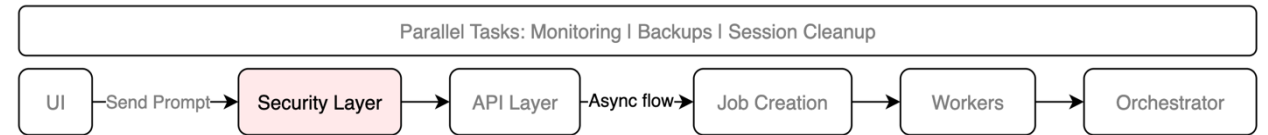
- OAuth 2.0 login & JWT-based authentication
- Tenant, role, and permission check on every request (RBAC)

Role	agent-runs	jobs	tools	tenants	Graph (write)
admin	CRUD	CRUD	CRUD	CRUD	✓
user	CR	CRD	R	X	X



SECURITY LAYER

8 protection layers before business logic



Request Protection Layers

1. CORS Handler

Trusted origins only

2. Trace Context

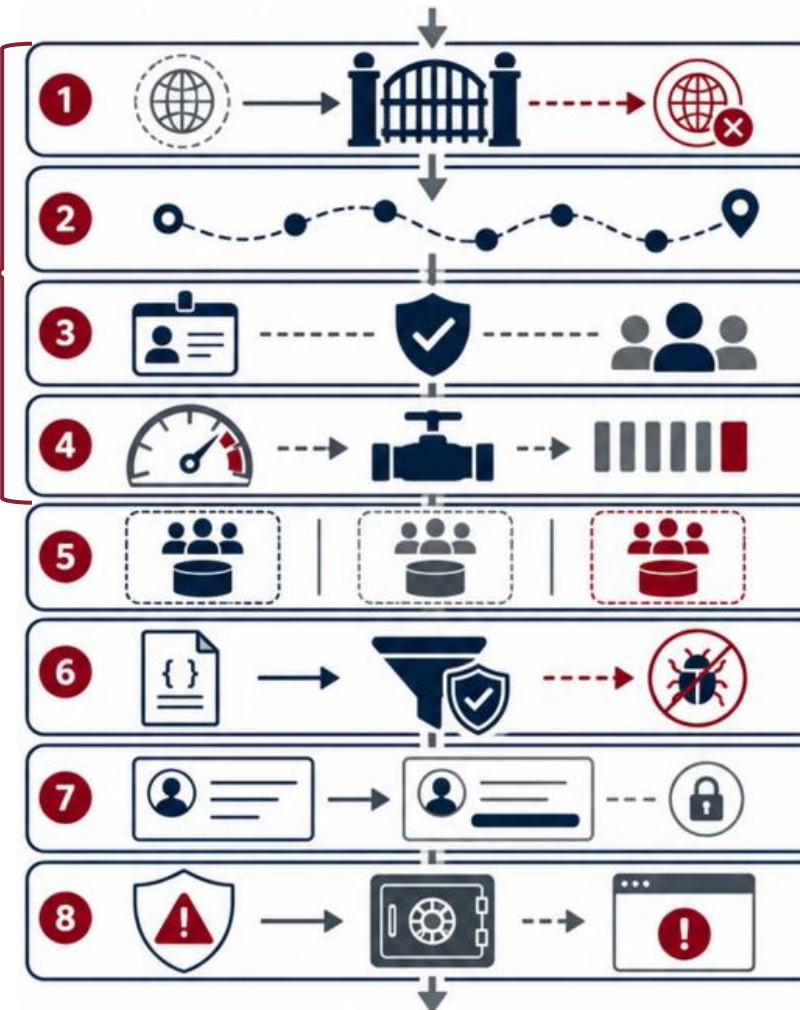
End-to-end request tracking

3. JWT Auth + RBAC

Verified identity and permissions

4. Rate Limiter

Abuse and overload protection



5. Tenant Resolver

Multi-tenant data isolation

6. Input Guard

Request validation and injection blocking

7. Output Guard

Response sanitization and PII redaction

8. Error Handler

Safe, standardized error responses

API LAYER - ROUTERS

76 endpoints

Core Platform APIs

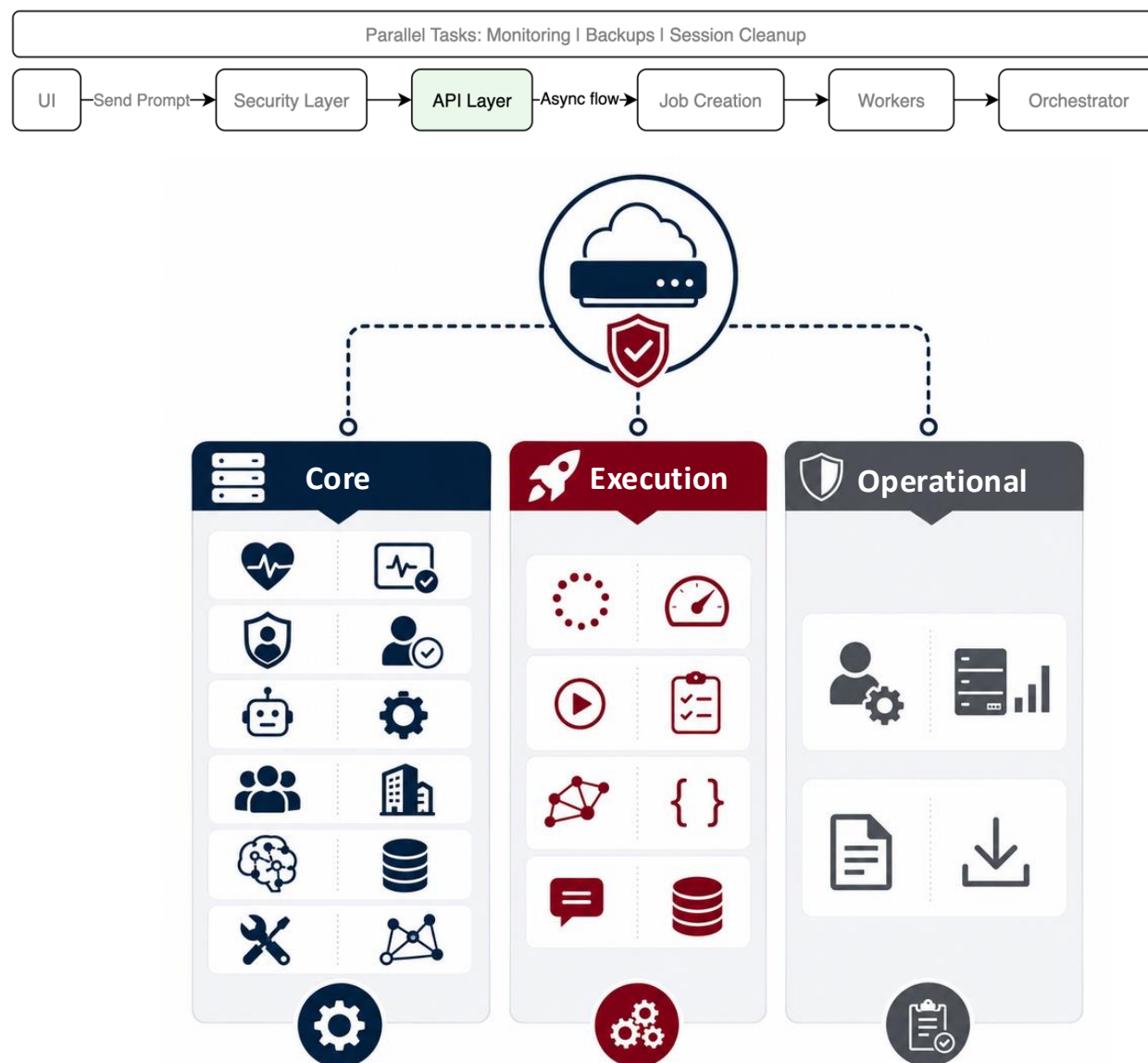
- **Health checks** — service and dependency status
- **Authentication** — token validation and user identity
- **Agents** — agent definitions and configuration
- **Tenants** — tenant settings, limits, and feature control
- **Models** — LLM provider and model management
- **Tools** — MCP tool discovery and invocation

Execution APIs

- **Jobs** — async tasks, progress streaming, cancellation
- **Agent runs** — run lifecycle, status, results, cancellation
- **Graph** — Cypher and NL→Cypher execution
- **Sessions** — conversation and runtime context

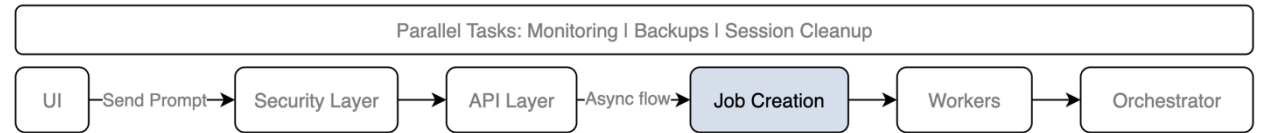
Operational APIs

- **Admin** — maintenance, cache control, system stats
- **Export** — runs, job results, and audit logs



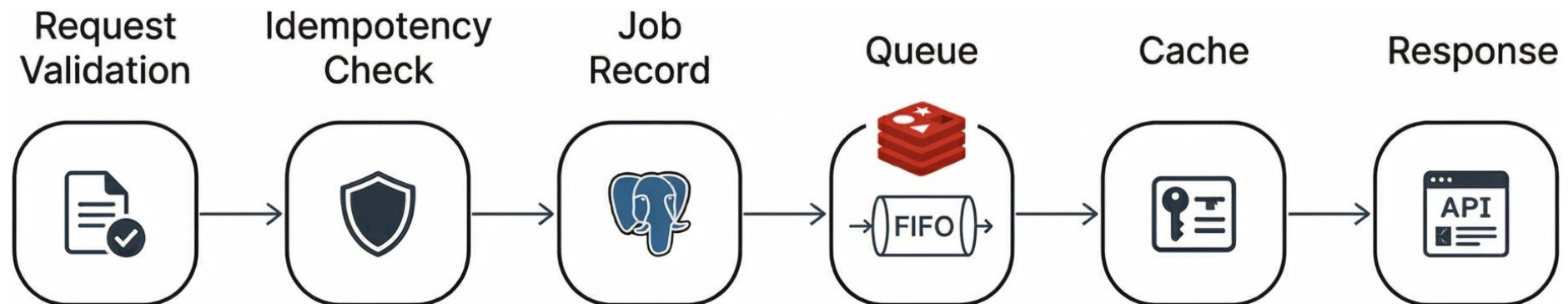
JOB CREATION

Safe intake for long-running tasks



Creation Flow

1. **Request validation** — required fields and payload structure
2. **Idempotency check** — duplicate prevention on retries
3. **Durable job record** — PostgreSQL job with status = queued
4. **Redis queueing** — job ID pushed for worker execution
5. **Fast retry cache** — idempotency key → job ID
6. **Async response** — HTTP 202 Accepted with job ID and status



WORKERS

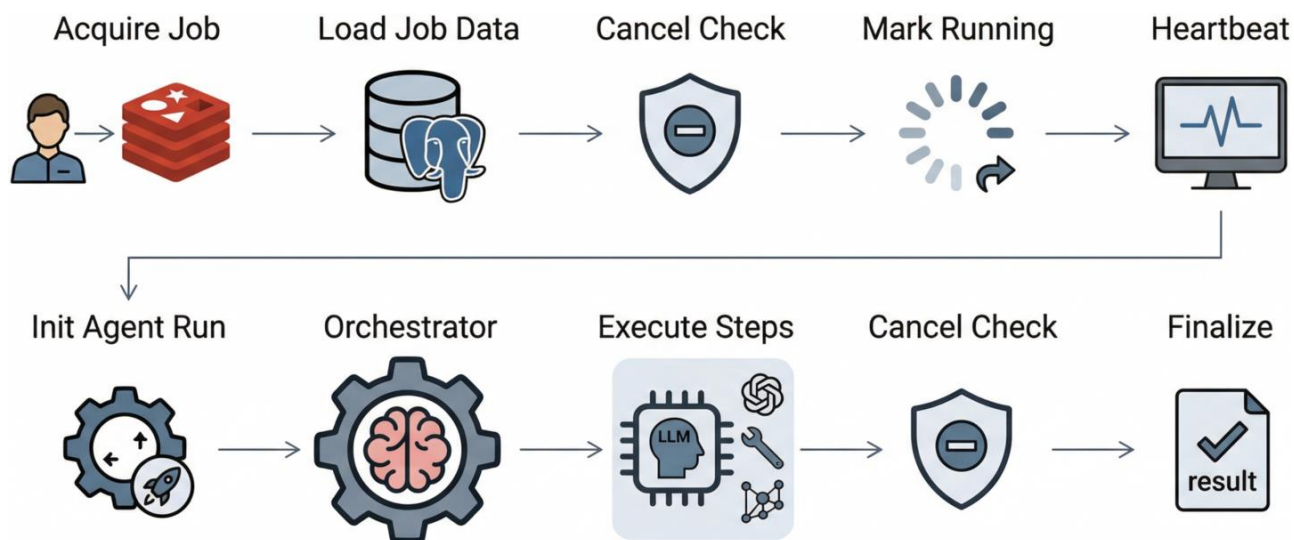
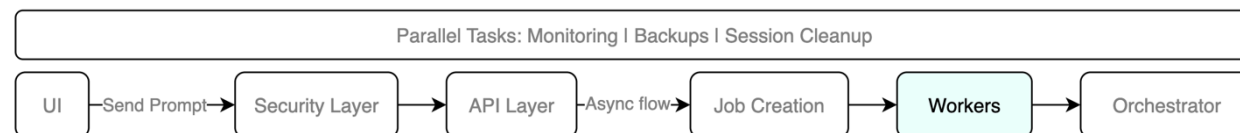
Reliable job execution

Worker Lifecycle

1. **Acquire queued job** → exclusive Redis ownership
2. **Load job metadata** → PostgreSQL as source of truth
3. **Check cancellation** → skip safely if already cancelled
4. **Mark as running** → auditable status transition
5. **Start heartbeat** → stuck-worker detection

Agent Run Execution

6. **Initialize AgentRun** → link job and run records
7. **Run orchestrator** → step-by-step workflow execution
8. **Execute and persist LLM / tool / graph steps**
9. **Stream live updates** → progress events to the UI
10. **Finalize safely** → output guards, metrics, final status



ORCHESTRATOR (Part 1/3)

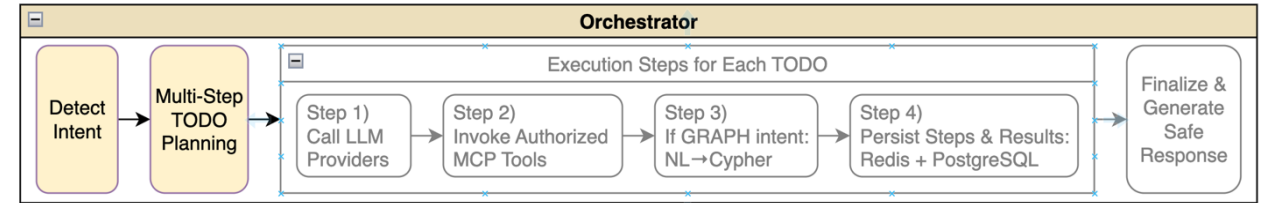
Intent detection & Multi-step planning

Phase 1 — Intent Classification

- 1 LLM call for classification
- Intent Types:
 - **CHAT** — conversational response
 - **GRAPH** — NL→Cypher query
 - **SECURITY** — permission and access questions
 - **ADMIN** — administrative actions
 - **DANGEROUS** — refused safely
 - **EXPLAIN** — analysis without execution

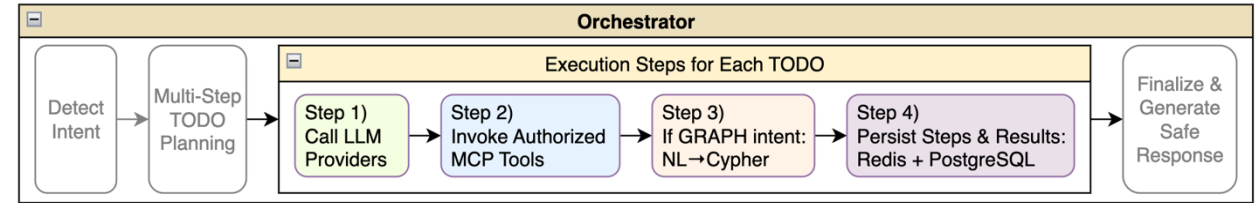
Phase 2 — TODO Planning

- Multi-step task decomposition
- Context-aware planning
- Schema and tool awareness
- Structured action sequence
- Example flow: query → analyze → summarize



ORCHESTRATOR (Part 2/3)

Step Execution



Phase 3: Step-by-Step Execution

1. Call LLM providers

- Planning, checking, and response generation
- Retries, fallbacks, and circuit breakers

2. Invoke MCP tools

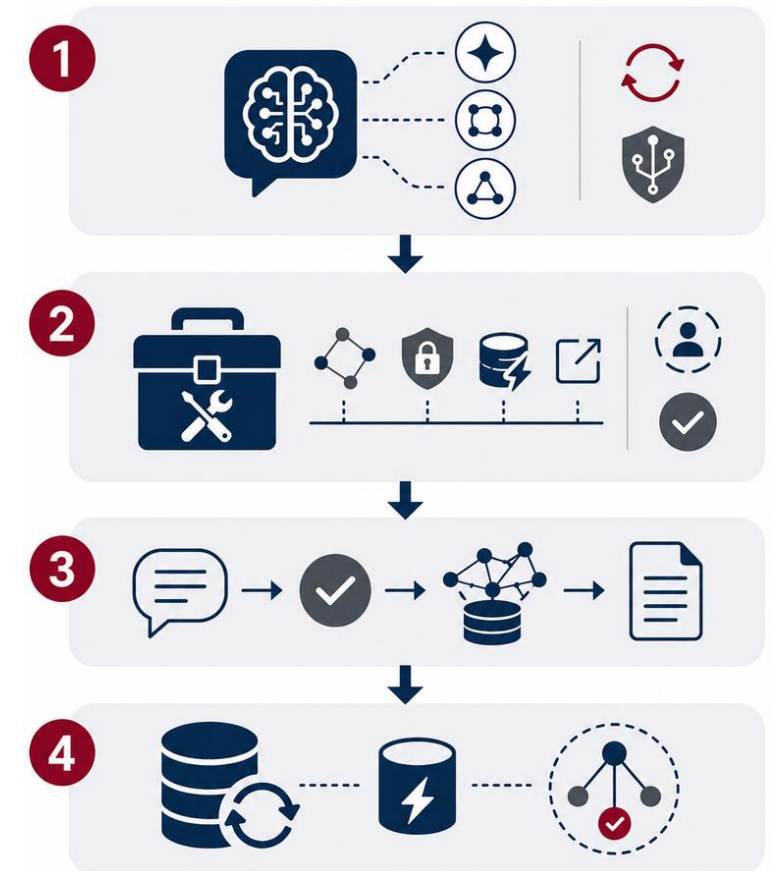
- Graph, security, cache, export, and system actions
- Session-aware tool execution
- Permission checks and runtime context

3. Graph pipeline

- NL→Cypher generation, validation, execution, and summarization

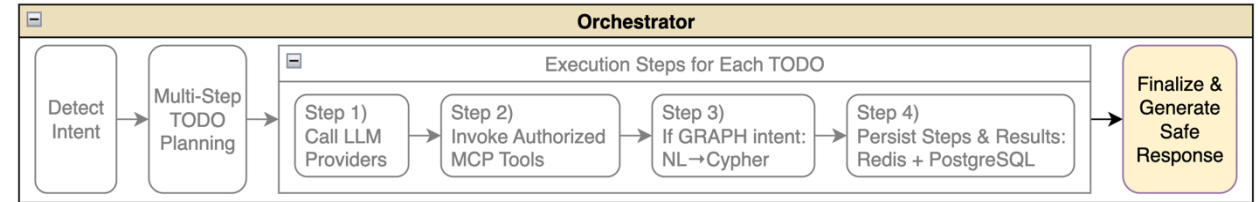
4. Progress persistence

- PostgreSQL for traceability and recovery
- Redis for caching, runtime coordination, and cancellation checks



ORCHESTRATOR (Part 3/3)

Finalization & Safe Response Generation



Phase 4: Final Output Pipeline - structured, persisted, safe, and observable

- **Response assembly**
Completed step outputs combined into one answer
- **Safety checks**
Sensitive data removal and unsafe content filtering
- **Fallback handling**
Cached or default response when needed
- **Final state persistence**
Status, output, and metrics saved to PostgreSQL
- **Output normalization**
Consistent text and optional JSON format
- **Observability data**
Metrics and traces for dashboards, debugging, and monitoring

```
{  
  "goal": "Find collaborating institutions",  
  "steps": [step1, step2, step3],  
  "outputs": ["Found 5 institutions..."],  
  "todos": [completed_todo1, completed_todo2],  
  "metrics": { "duration_ms": 2340, "tokens": 1850 }  
}
```

LLM PROVIDERS

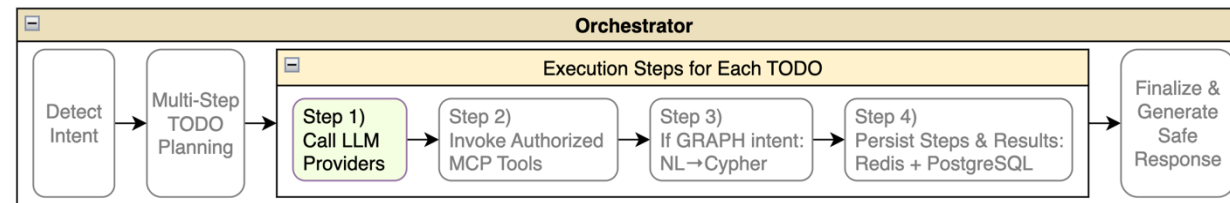
Model Selection

Provider Options

- **Local models** — Ollama, self-hosted open models
- **Cloud models** — OpenAI, Azure OpenAI, compatible providers
- **Provider flexibility** — same platform logic, different model backends
- **No vendor lock-in** — switch providers without changing orchestration logic

Why It Matters

- **Data control**
- **Lower cost options**
- **Offline or private deployment**
- **Managed cloud quality**
- **Enterprise compliance**
- **Regional hosting**



Model Selection Hierarchy:

Request-level override

↓ (if not set)

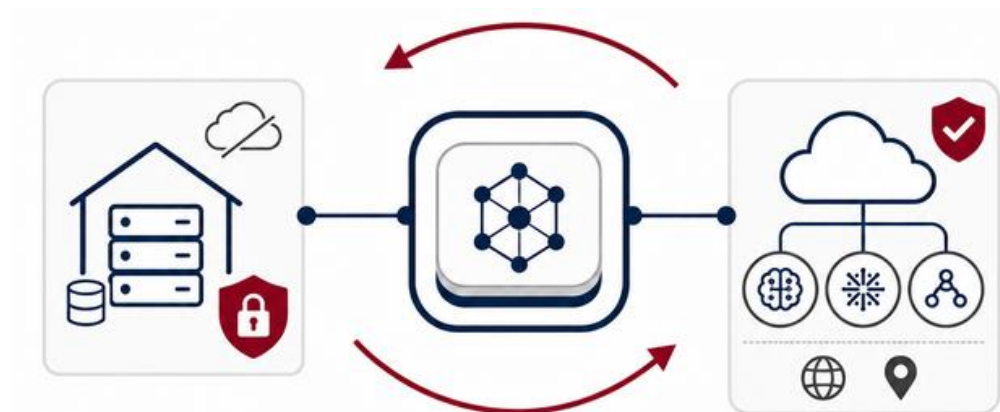
Session-level default

↓ (if not set)

Tenant-level default

↓ (if not set)

Global platform default



MCP RUNTIME & TOOLS

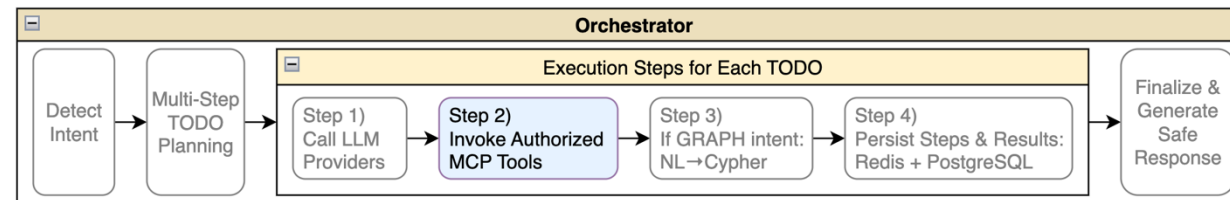
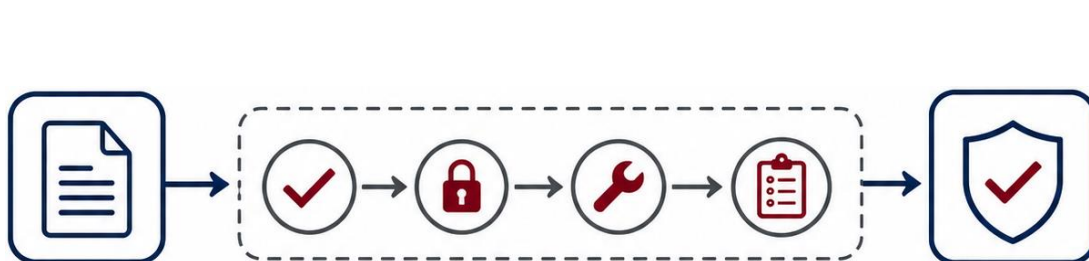
34 tools across 12 categories

Core Runtime Ideas

- **Tool registry** — schemas, scopes, limits
- **Permission checks** — RBAC before execution
- **Schema validation** — structured and safe inputs
- **Audited execution** — every tool call recorded
- **Controlled results** — safe outputs returned to the agent

Tool Invocation Flow

Request → Validate → Authorize → Execute → Audit → Return



Main Tool Categories

- **Graph** — Memgraph, schema, Cypher
- **Security** — identity, permissions, access checks
- **System** — health, metrics, configuration
- **Catalog** — tools, resources, schemas
- **Model** — list, inspect, warm up, switch
- **Agent** — context, history, session state
- **Analytics** — aggregations and visualization data
- **Admin** — restricted operations
- **ETL / CRUD / Export** — import, update, export



DATA LAYER

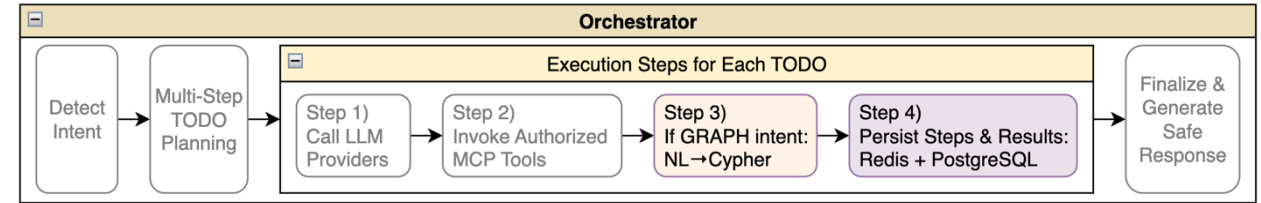
REDIS, POSTGRESQL, MEMGRAPH

Redis — Real-Time Runtime State

- Cache and sessions
- Tenant and model defaults
- Rate limits
- Job queues and progress events
- Cancellation and idempotency
- Circuit-breaker state

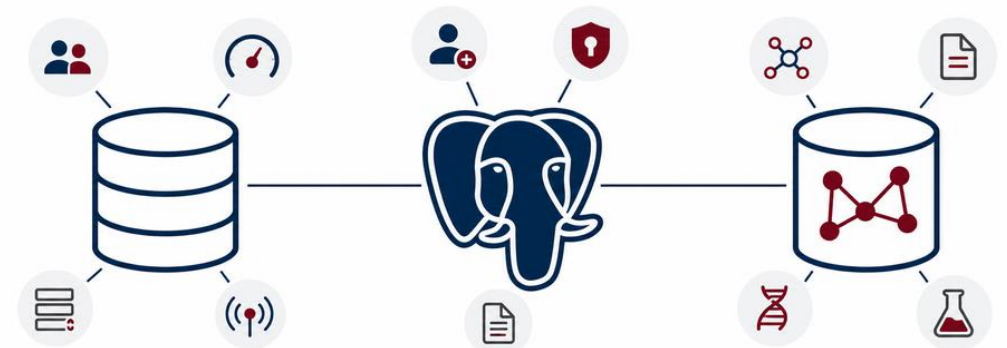
PostgreSQL — Durable Platform State

- Tenants, sessions, runs, steps, jobs
- Job events and tool invocations
- Model and provider configuration
- Encrypted secrets
- Audit logs
- Idempotency records



Memgraph — Bioinformatics Graph State

- Connected graph entities
- Users, institutions, tasks, files, datasets
- Genes, proteins, pathways, workflows, results
- Relationships: WORKS_AT, RUNS, INPUT, OUTPUT
- Cypher queries and graph metadata



BACKGROUND TASKS

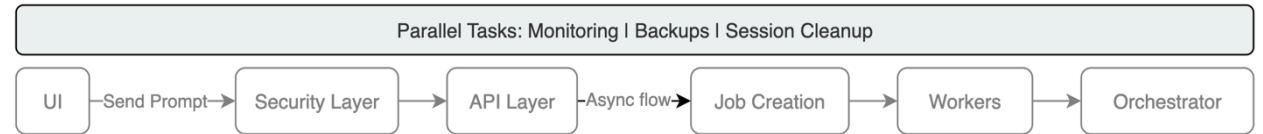
Monitoring | Backups | Session Cleanup

Background Automation

- APScheduler
- Health checks
- Session and cache cleanup
- Completed job cleanup
- Daily backups
- Provider monitoring
- Circuit-breaker updates

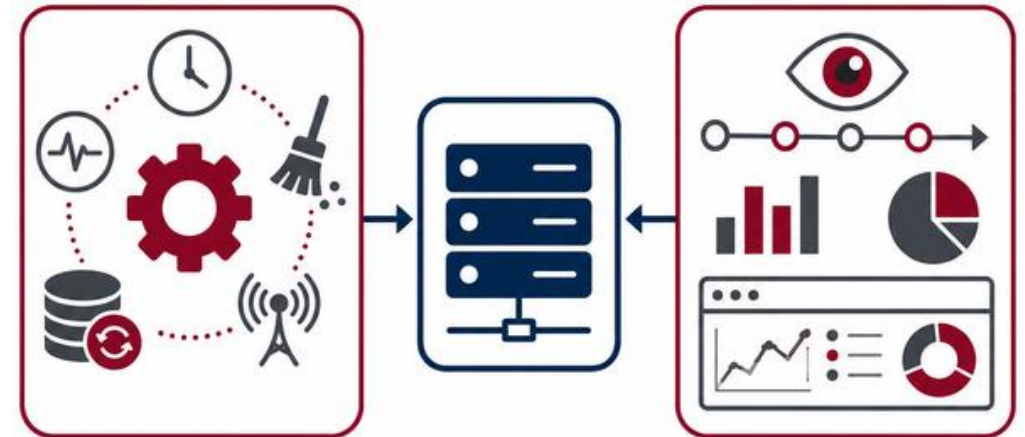
Observability Stack

- **OpenTelemetry** — end-to-end traces
- **Prometheus** — platform and worker metrics
- **Grafana** — dashboards and operational views



Operational Value

- System health visibility
- Slow request diagnosis
- Failed tool detection
- Provider health monitoring
- Queue depth and cost tracking
- Recovery and debugging support



VALIDATION & RESULTS

End-to-end verification

How was the platform validated?

- 1,000+ automated tests for core logic and E2E workflows
- Verified lifecycle, async execution, streaming, cancellation, RBAC, tenancy, and guards
- Full NL→Graph path tested:
Natural language → GRAPH intent → Cypher generation → safety validation → Memgraph execution → final answer

Example:

"Count Blast nodes grouped by presence of blast_version"

1. Classify intent → GRAPH

2. Generate Cypher→

`MATCH (b:Blast) RETURN b.blast_version IS NOT NULL AS has_blast_version, count(b) AS count ORDER BY has_blast_version DESC`

3. Validate → ✓ Read-only ✓ Tenant ✓ Depth ✓ Timeout

4. Cypher Execution → Returns 1 row

5. Summarize → "In total, there are 39 Blast nodes in the database. All containing the blast_version attribute."

COMPARISON WITH STATE OF THE ART

Why Cineca Agentic Platform

Platform	Full Stack	Agentic Planning	Governed Tools	Enterprise Security	Graph NL→Cypher	Observability	Summary
CAP	✓	✓	✓	✓	✓	✓	Integrated production platform
Temporal	✗	✗	✗	✓	✗	✓	strong workflows, not agent platforms
LangGraph	✗	✓	~	✗	~	~	Strong agent framework, limited platform controls
OpenAI Agents SDK	✗	✓	~	✗	~	~	Fast agent development, Lacks integrated security layer
Semantic Kernel	✗	✓	~	~	~	~	Enterprise AI middleware, Partial support across platform capabilities
n8n	✓	~	~	~	✗	~	Visual workflow automation, Not designed for governed agent orchestration

Main takeaway:

- CAP combines agent execution, tools, jobs, graph access, security, and observability in one platform.

✓ = Native support
~ = Partial / requires customization
✗ = Not supported

CONCLUSION + FUTURE WORK

Closed the production gap: From natural language to safe, traceable graph actions

Key Contribution

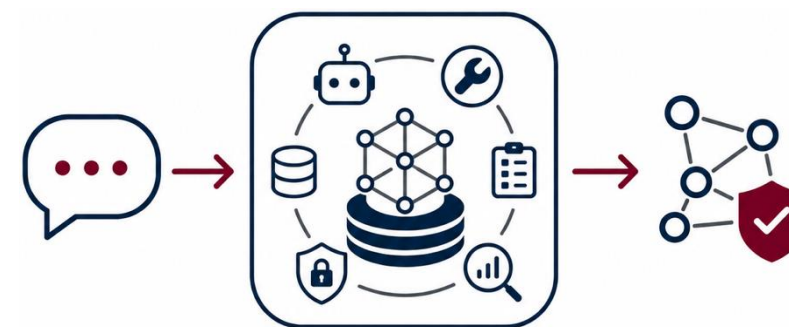
- An extensible multi-tenant AI orchestration platform that unifies agents, tools, jobs, graph access, security, and observability.

What Was Delivered

- **Full-stack agentic platform** — UI, APIs, jobs, workers, orchestration
- **Secure NL-to-action execution** — controlled agent runs and safe NL→Cypher
- **Governed MCP tools** — permission-checked, validated, and audited execution
- **Enterprise security** — JWT, RBAC, tenancy, input/output guards
- **Production readiness** — retries, checkpoints, observability, monitoring
- **Model flexibility** — OpenAI, Azure, Ollama, and compatible providers

Future Work

- Lightweight Ask Mode
- Broader prompt handling
- Stronger checkpointing and deterministic re-runs
- Smarter scheduling with priorities, quotas, and backpressure

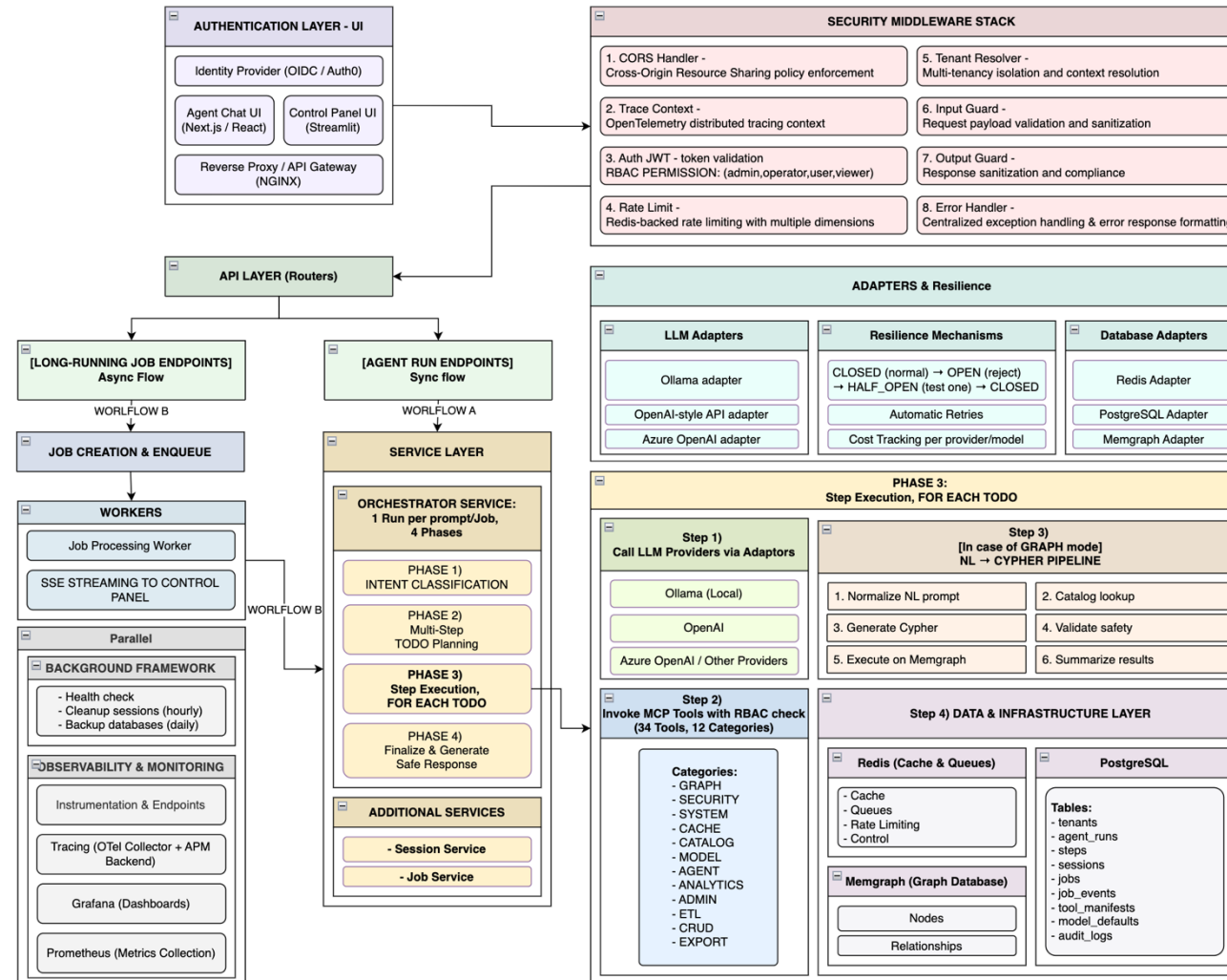


THANK YOU FOR YOUR ATTENTION

Feel free to ask any question

BACKUP SLIDES

Backup Slides - Detailed System Architecture



ADAPTERS + RESILIENCE

Flexible integrations with automatic failure handling

Swappable Adapters

- Vendor-independent business logic
- Unified LLM interface
- Ollama, OpenAI, Azure, stub/demo providers
- Unified database access
- Redis, PostgreSQL, Memgraph adapters
- Backend changes without orchestrator rewrites

Resilience Mechanisms

- Retries with backoff
- Circuit breakers
- Provider fallback
- Fail-fast behavior
- Closed → Open → Half-open recovery

Operational Tracking

- Token usage
- Cost tracking
- Latency measurement
- Monitoring and budgeting

